



OVP Guide to Using Processor Models

Model specific information for ARM_Cortex-R4F

Imperas Software Limited
Imperas Buildings, North Weston
Thame, Oxfordshire, OX9 2HA, U.K.
docs@imperas.com



Author	Imperas Software Limited
Version	20250716.0
Filename	OVP_Model_Specific_Information_arm_Cortex-R4F.pdf
Created	16 July 2025
Status	OVP Standard Release

Copyright Notice

Copyright (c) 2024 Imperas Software Limited. All rights reserved. This software and documentation contain information that is the property of Imperas Software Limited. The software and documentation are furnished under a license agreement and may be used or copied only in accordance with the terms of the license agreement. No part of the software and documentation may be reproduced, transmitted, or translated, in any form or by any means, electronic, mechanical, manual, optical, or otherwise, without prior written permission of Imperas Software Limited, or as expressly provided by the license agreement.

Right to Copy Documentation

The license agreement with Imperas permits licensee to make copies of the documentation for its internal use only. Each copy shall include all copyrights, trademarks, service marks, and proprietary rights notices, if any.

Destination Control Statement

All technical data contained in this publication is subject to the export control laws of the United States of America. Disclosure to nationals of other countries contrary to United States law is prohibited. It is the readers responsibility to determine the applicable regulations and to comply with them.

Disclaimer

IMPERAS SOFTWARE LIMITED, AND ITS LICENSORS MAKE NO WARRANTY OF ANY KIND, EXPRESS OR IMPLIED, WITH REGARD TO THIS MATERIAL, INCLUDING, BUT NOT LIMITED TO, THE IMPLIED WARRANTIES OF MERCHANTABILITY AND FITNESS FOR A PARTICULAR PURPOSE.

Model Release Status

This model is released as part of OVP releases and is included in OVPworld packages. Please visit OVPworld.org.

Contents

1	Overview	1
1.1	Description	1
1.2	Licensing	1
1.3	Limitations	2
1.4	Verification	2
1.5	Features	3
1.5.1	Core Features	3
1.5.2	Memory System	3
1.5.3	Advanced SIMD and Floating-Point Features	3
1.6	Debug Mask	3
1.7	AArch32 Unpredictable Behavior	4
1.7.1	Equal Target Registers	4
1.7.2	Floating Point Load/Store Multiple Lists	4
1.7.3	Floating Point VLD[2-4]/VST[2-4] Range Overflow	4
1.7.4	If-Then (IT) Block Constraints	4
1.7.5	Use of R13	5
1.7.6	Use of R15	5
1.7.7	Unpredictable Instructions in Some Modes	5
1.8	Integration Support	5
1.8.1	Halt Reason Introspection	5
1.8.2	Automatic WFI Wake Up	6
1.8.3	System Register Access Monitor	6
1.8.4	System Register Implementation	6
2	Configuration	7
2.1	Location	7
2.2	GDB Path	7
2.3	Semi-Host Library	7
2.4	Processor Endian-ness	7
2.5	QuantumLeap Support	7
2.6	Processor ELF code	7
3	All Variants in this model	8
4	Bus Master Ports	12
5	Bus Slave Ports	13

6	Net Ports	14
7	FIFO Ports	15
8	Formal Parameters	16
8.1	Parameter values and limits	19
9	Execution Modes	22
10	Exceptions	23
11	Hierarchy of the model	24
11.1	Level 1: CPU	24
12	Model Commands	25
12.1	Level 1: CPU	25
12.1.1	debugflags	25
12.1.2	isync	25
12.1.3	itrace	25
12.1.4	listSysRegsAA32	26
12.1.4.1	Argument description	26
13	Registers	27
13.1	Level 1: CPU	27
13.1.1	Core	27
13.1.2	Control	27
13.1.3	User	27
13.1.4	FIQ	28
13.1.5	IRQ	28
13.1.6	Supervisor	28
13.1.7	Undefined	28
13.1.8	Abort	29
13.1.9	SIMD_VFP	29
13.1.10	SIMD_VFP_SYS	29
13.1.11	Coprocessor_32_bit	30
13.1.12	Integration_support	31

Chapter 1

Overview

This document provides the details of an OVP Fast Processor Model variant.

OVP Fast Processor Models are written in C and provide a C API for use in C based platforms. The models also provide a native interface for use in SystemC TLM2 platforms.

The models are written using the OVP VMI API that provides a Virtual Machine Interface that defines the behavior of the processor. The VMI API makes a clear line between model and simulator allowing very good optimization and world class high speed performance. Most models are provided as a binary shared object and also as source. This allows the download and use of the model binary or the use of the source to explore and modify the model.

The models are run through an extensive QA and regression testing process and most model families are validated using technology provided by the processor IP owners. There is a companion document (OVP Guide to Using Processor Models) which explains the general concepts of OVP Fast Processor Models and their use. It is downloadable from the OVPworld website documentation pages.

1.1 Description

ARM Processor Model

1.2 Licensing

Usage of binary model under license governing simulator usage.

Note that for models of ARM CPUs the license includes the following terms:

Licensee is granted a non-exclusive, worldwide, non-transferable, revocable licence to:

If no source is being provided to the Licensee: use and copy only (no modifications rights are granted) the model for the sole purpose of designing, developing, analyzing, debugging, testing, verifying, validating and optimizing software which: (a) (i) is for ARM based systems; and (ii) does not incorporate the ARM Models or any part thereof; and (b) such ARM Models may not be used

to emulate an ARM based system to run application software in a production or live environment.

If source code is being provided to the Licensee: use, copy and modify the model for the sole purpose of designing, developing, analyzing, debugging, testing, verifying, validating and optimizing software which: (a) (i) is for ARM based systems; and (ii) does not incorporate the ARM Models or any part thereof; and (b) such ARM Models may not be used to emulate an ARM based system to run application software in a production or live environment.

In the case of any Licensee who is either or both an academic or educational institution the purposes shall be limited to internal use.

Except to the extent that such activity is permitted by applicable law, Licensee shall not reverse engineer, decompile, or disassemble this model. If this model was provided to Licensee in Europe, Licensee shall not reverse engineer, decompile or disassemble the Model for the purposes of error correction.

The License agreement does not entitle Licensee to manufacture in silicon any product based on this model.

The License agreement does not entitle Licensee to use this model for evaluating the validity of any ARM patent.

Source of model available under separate Imperas Software License Agreement.

1.3 Limitations

Instruction pipelines are not modeled in any way. All instructions are assumed to complete immediately. This means that instruction barrier instructions (e.g. ISB, CP15ISB) are treated as NOPs, with the exception of any undefined instruction behavior, which is modeled. The model does not implement speculative fetch behavior. The branch cache is not modeled.

Caches and write buffers are not modeled in any way. All loads, fetches and stores complete immediately and in order, and are fully synchronous (as if the memory was of Strongly Ordered or Device-nGnRnE type). Data barrier instructions (e.g. DSB, CP15DSB) are treated as NOPs, with the exception of any undefined instruction behavior, which is modeled. Cache manipulation instructions are implemented as NOPs, with the exception of any undefined instruction behavior, which is modeled.

Real-world timing effects are not modeled: all instructions are assumed to complete in a single cycle.

Performance Monitors are implemented as a register interface only except for the cycle counter, which is implemented assuming one instruction per cycle.

1.4 Verification

Models have been extensively tested by Imperas. ARM Cortex models have been successfully used by customers to simulate SMP Linux, Ubuntu Desktop, VxWorks and ThreadX on Xilinx Zynq virtual platforms.

1.5 Features

The precise set of implemented features in the model is defined by ID registers. Use overrides to modify these if required (for example `override_PFR0` or `override_AA64PFR0_EL1`).

1.5.1 Core Features

Thumb-2 instructions are supported.

Trivial Jazelle extension is implemented.

Vectored Interrupt Controller Port (VIC port) is implemented.

1.5.2 Memory System

PMSA address translation is implemented with a unified MPU with 8 regions. Use parameter “`override_MPUIR`” to specify a different MPU size if required.

1 ATCM is implemented.

1 BTCM is implemented.

For each TCM, a corresponding bus port is present. If parameter “`useInternalTCMs`” is `True`, TCMs are modelled internally and the port is a target port, allowing TCM contents to be examined and updated externally. Otherwise, if parameter “`useInternalTCMs`” is `False`, the corresponding port is an initiator port to which an external TCM memory model must be connected.

1.5.3 Advanced SIMD and Floating-Point Features

VFP instructions are implemented.

The model implements trapped exceptions if `FPTrap` is set to 1 in `MVFR0` (for `AArch32`) or `MVFR0_EL1` (for `AArch64`). When floating point exception traps are taken, cumulative exception flags are not updated (in other words, cumulative flag state is always the same as prior to instruction execution, even for SIMD instructions). When multiple enabled exceptions are raised by a single floating point operation, the exception reported is the one in least-significant bit position in `FPSCR` (for `AArch32`) or `FPCR` (for `AArch64`). When multiple enabled exceptions are raised by different SIMD element computations, the exception reported is selected from the lowest-index-number SIMD operation. Contact Imperas if requirements for exception reporting differ from these.

Trapped exceptions not are implemented in this variant (`FPTrap=0`)

1.6 Debug Mask

It is possible to enable model debug features in various categories. This can be done statically using the “`override_debugMask`” parameter, or dynamically using the “`debugflags`” command. Enabled debug features are specified using a bitmask value, as follows:

Value 0x004: enable debugging of MMU/MPU mappings.

Value 0x080: enable debugging of all system register accesses.

Value 0x100: enable debugging of all traps of system register accesses.

Value 0x200: enable verbose debugging of other miscellaneous behavior (for example, the reason why a particular instruction is undefined).

Value 0x400: enable debugging of Performance Monitor timers

All other bits in the debug bitmask are reserved and must not be set to non-zero values.

1.7 AArch32 Unpredictable Behavior

Many AArch32 instruction behaviors are described in the ARM ARM as CONSTRAINED UNPREDICTABLE. This section describes how such situations are handled by this model.

1.7.1 Equal Target Registers

Some instructions allow the specification of two target registers (for example, double-width SMULL, or some VMOV variants), and such instructions are CONSTRAINED UNPREDICTABLE if the same target register is specified in both positions. In this model, such instructions are treated as UNDEFINED.

1.7.2 Floating Point Load/Store Multiple Lists

Instructions that load or store a list of floating point registers (e.g. VSTM, VLDM, VPUSH, VPOP) are CONSTRAINED UNPREDICTABLE if either the uppermost register in the specified range is greater than 32 or (for 64-bit registers) if more than 16 registers are specified. In this model, such instructions are treated as UNDEFINED.

1.7.3 Floating Point VLD[2-4]/VST[2-4] Range Overflow

Instructions that load or store a fixed number of floating point registers (e.g. VST2, VLD2) are CONSTRAINED UNPREDICTABLE if the upper register bound exceeds the number of implemented floating point registers. In this model, these instructions load and store using modulo 32 indexing (consistent with AArch64 instructions with similar behavior).

1.7.4 If-Then (IT) Block Constraints

Where the behavior of an instruction in an if-then (IT) block is described as CONSTRAINED UNPREDICTABLE, this model treats that instruction as UNDEFINED.

1.7.5 Use of R13

In architecture variants before ARMv8, use of R13 was described as **CONSTRAINED UNPREDICTABLE** in many circumstances. From ARMv8, most of these situations are no longer considered unpredictable. This model allows R13 to be used like any other GPR, consistent with the ARMv8 specification.

1.7.6 Use of R15

Use of R15 is described as **CONSTRAINED UNPREDICTABLE** in many circumstances. This model allows such use to be configured using the parameter “unpredictableR15” as follows:

Value “undefined”: any reference to R15 in such a situation is treated as **UNDEFINED**;

Value “nop”: any reference to R15 in such a situation causes the instruction to be treated as a **NOP**;

Value “raz_wi”: any reference to R15 in such a situation causes the instruction to be treated as a **RAZ/WI** (that is, R15 is read as zero and write-ignored);

Value “execute”: any reference to R15 in such a situation is executed using the current value of R15 on read, and writes to R15 are allowed (but are not interworking).

Value “assert”: any reference to R15 in such a situation causes the simulation to halt with an assertion message (allowing any such unpredictable uses to be easily identified).

In this variant, the default value of “unpredictableR15” is “undefined”.

1.7.7 Unpredictable Instructions in Some Modes

Some instructions are described as **CONSTRAINED UNPREDICTABLE** in some modes only (for example, MSR accessing SPSR is **CONSTRAINED UNPREDICTABLE** in User and System modes). This model allows such use to be configured using the parameter “unpredictableModal”, which can have values “undefined” or “nop”. See the previous section for more information about the meaning of these values.

In this variant, the default value of “unpredictableModal” is “nop”.

1.8 Integration Support

This model implements a number of non-architectural pseudo-registers and other features to facilitate integration.

1.8.1 Halt Reason Introspection

An artifact register **HaltReason** can be read to determine the reason or reasons that a processor is halted. This register is a bitfield, with the following encoding: bit 0 indicates the processor

has executed a wait-for-event (WFE) instruction; bit 1 indicates the processor has executed a wait-for-interrupt (WFI) instruction; and bit 2 indicates the processor is held in reset.

1.8.2 Automatic WFI Wake Up

A simulation environment can cause a core to wake up from the WFI state by writing to the artifact register WakeWFINow.

1.8.3 System Register Access Monitor

If parameter “enableSystemMonitorBus” is True, an artifact 32-bit bus “SystemMonitor” is enabled for each PE. Every system register read or write by that PE is then visible as a read or write on this artifact bus, and can therefore be monitored using callbacks installed in the client environment (use opBusReadMonitorAdd/opBusWriteMonitorAdd or icmAddBusReadCallback-/icmAddBusWriteCallback, depending on the client API). The format of the address on the bus is as follows:

bits 31:26 - zero

bit 25 - 1 if AArch64 access, 0 if AArch32 access

bit 24 - 1 if non-secure access, 0 if secure access

bits 23:20 - CRm value

bits 19:16 - CRn value

bits 15:12 - op2 value

bits 11:8 - op1 value

bits 7:4 - op0 value (AArch64) or coprocessor number (AArch32)

bits 3:0 - zero

As an example, to view non-secure writes to writes to CNTFRQ_EL0 in AArch64 state, install a write monitor on address range 0x020e0330:0x020e0333.

1.8.4 System Register Implementation

If parameter “enableSystemBus” is True, an artifact 32-bit bus “System” is enabled for each PE. Slave callbacks installed on this bus can be used to implement modified system register behavior (use opBusSlaveNew or icmMapExternalMemory, depending on the client API). The format of the address on the bus is the same as for the system monitor bus, described above.

Chapter 2

Configuration

2.1 Location

This model's VLVN is `arm.ovpworld.org/processor/arm/1.0`.

The model source is usually at:

`$IMPERAS_HOME/ImperasLib/source/arm.ovpworld.org/processor/arm/1.0`

The model binary is usually at:

`$IMPERAS_HOME/lib/$IMPERAS_ARCH/ImperasLib/arm.ovpworld.org/processor/arm/1.0`

2.2 GDB Path

The default GDB for this model is: `$IMPERAS_HOME/lib/$IMPERAS_ARCH/gdb/arm-none-eabi-gdb`.

2.3 Semi-Host Library

The default semi-host library file is `arm.ovpworld.org/semihosting/armNewlib/1.0`

2.4 Processor Endian-ness

This is a LITTLE endian model.

2.5 QuantumLeap Support

This processor is qualified to run in a QuantumLeap enabled simulator.

2.6 Processor ELF code

The ELF code supported by this model is: `0x28`.

Chapter 3

All Variants in this model

This model has these variants

Variant	Description
ARMv4T	
ARMv4xM	
ARMv4	
ARMv4TxM	
ARMv5xM	
ARMv5	
ARMv5TxM	
ARMv5T	
ARMv5TExP	
ARMv5TE	
ARMv5TEJ	
ARMv6	
ARMv6K	
ARMv6T2	
ARMv6KZ	
ARMv7	
ARM7TDMI	
ARM7EJ-S	
ARM720T	
ARM920T	
ARM922T	
ARM926EJ-S	
ARM940T	
ARM946E	
ARM966E	
ARM968E-S	
ARM1020E	
ARM1022E	
ARM1026EJ-S	
ARM1136J-S	
ARM1156T2-S	

ARM1176JZ-S	
Cortex-R4	
Cortex-R4F	(described in this document)
Cortex-R5MPx1	
Cortex-R5MPx2	
Cortex-R8MPx1	
Cortex-R8MPx2	
Cortex-R8MPx3	
Cortex-R8MPx4	
Cortex-R52MPx1	
Cortex-R52MPx2	
Cortex-R52MPx3	
Cortex-R52MPx4	
Cortex-R52+MPx1	
Cortex-R52+MPx2	
Cortex-R52+MPx3	
Cortex-R52+MPx4	
Cortex-R82MPx1	
Cortex-R82MPx2	
Cortex-R82MPx3	
Cortex-R82MPx4	
Cortex-R82MPx5	
Cortex-R82MPx6	
Cortex-R82MPx7	
Cortex-R82MPx8	
Cortex-A5UP	
Cortex-A5MPx1	
Cortex-A5MPx2	
Cortex-A5MPx3	
Cortex-A5MPx4	
Cortex-A8	
Cortex-A9UP	
Cortex-A9MPx1	
Cortex-A9MPx2	
Cortex-A9MPx3	
Cortex-A9MPx4	
Cortex-A7UP	
Cortex-A7MPx1	
Cortex-A7MPx2	
Cortex-A7MPx3	
Cortex-A7MPx4	
Cortex-A15UP	
Cortex-A15MPx1	
Cortex-A15MPx2	
Cortex-A15MPx3	

Cortex-A15MPx4	
Cortex-A17MPx1	
Cortex-A17MPx2	
Cortex-A17MPx3	
Cortex-A17MPx4	
AArch32	
AArch64	
Cortex-A32MPx1	
Cortex-A32MPx2	
Cortex-A32MPx3	
Cortex-A32MPx4	
Cortex-A35MPx1	
Cortex-A35MPx2	
Cortex-A35MPx3	
Cortex-A35MPx4	
Cortex-A53MPx1	
Cortex-A53MPx2	
Cortex-A53MPx3	
Cortex-A53MPx4	
Cortex-A55MPx1	
Cortex-A55MPx2	
Cortex-A55MPx3	
Cortex-A55MPx4	
Cortex-A57MPx1	
Cortex-A57MPx2	
Cortex-A57MPx3	
Cortex-A57MPx4	
Cortex-A65AEMPx1	
Cortex-A65AEMPx2	
Cortex-A65AEMPx3	
Cortex-A65AEMPx4	
Cortex-A72MPx1	
Cortex-A72MPx2	
Cortex-A72MPx3	
Cortex-A72MPx4	
Cortex-A73MPx1	
Cortex-A73MPx2	
Cortex-A73MPx3	
Cortex-A73MPx4	
Cortex-A75MPx1	
Cortex-A75MPx2	
Cortex-A75MPx3	
Cortex-A75MPx4	
Cortex-A76MPx1	
Cortex-A76MPx2	

Cortex-A76MPx3	
Cortex-A76MPx4	
Cortex-A78MPx1	
Cortex-A78MPx2	
Cortex-A78MPx3	
Cortex-A78MPx4	
MultiCluster	

Table 3.1: All Variants in this model

Chapter 4

Bus Master Ports

This model has these bus master ports.

Name	min	max	Connect?	Description
ATCM0	13	13	optional	ATCM
BTCM0	13	13	optional	BTCM
INSTRUCTION	32	32	mandatory	
DATA	32	32	optional	

Table 4.1: Bus Master Ports

Chapter 5

Bus Slave Ports

This model has no bus slave ports.

Chapter 6

Net Ports

This model has these net ports.

Name	Type	Connect?	Description
EVENTI	input	optional	Event input signal, active on rising edge
EVENTO	output	optional	Event output signal, active on rising edge
VINITHI	input	optional	Configure HIVECS mode (SCTLR.V)
CFGEE	input	optional	Configure exception endianness (SCTLR.EE)
CFGIE	input	optional	Configure instruction endianness (SCTLR.IE)
INTRAMA	input	optional	Configure ATCM enable bit (active high)
INTRAMB	input	optional	Configure BTCM enable bit (active high)
LOCZRAMA	input	optional	Configure ATCM/BTCM zero RAM
TEINIT	input	optional	Configure exception state at reset (SCTLR.TE)
CFGNMFI	input	optional	Configure non-maskable fast interrupts (SCTLR.NMFI)
reset	input	optional	Processor reset, active high
fiq	input	optional	FIQ interrupt, active high (negation of nFIQ)
irq	input	optional	IRQ interrupt, active high (negation of nIRQ)
sei	input	optional	System error interrupt, active on rising edge (negation of nSEI)
haltReason	output	optional	Indicates why core is halted
AXI.SLVERR	input	optional	AXI external abort type (DECERR=0, SLVERR=1)
VICACK	output	optional	VIC Port acknowledge (active high)
VICADDR	input	optional	VIC Port Address (32 bit value)
updateTimer	output	optional	Internal timer update

Table 6.1: Net Ports

Chapter 7

FIFO Ports

This model has no FIFO ports.

Chapter 8

Formal Parameters

Name	Type	Description
verbose	Boolean	Specify verbosity of output
suppressCPSWarnings	Boolean	Suppress duplicate warnings generated using ARM_CP_CPSI or ARM_CP_CPSD message identifiers
showHiddenRegs	Boolean	Show hidden registers during register tracing
UAL	Boolean	Disassemble using UAL syntax
enableVFPAAtReset	Boolean	Enable vector floating point (SIMD and VFP) instructions at reset. (Enables cp10/11 in CPACR and sets FPEXC.EN)
useInternalTCMs	Boolean	Enable internally-modeled TCM memories
enableSystemBus	Boolean	Add 32-bit artifact System bus port, allowing system registers to be externally implemented
enableSystemMonitorBus	Boolean	Add 32-bit artifact SystemMonitor bus port, allowing system register accesses to be externally monitored
compatibility	Enumeration	Specify compatibility mode
	ISA	
	gdb	
	nopSVC	
unpredictableR15	Enumeration	Specify behavior for UNPREDICTABLE uses of AArch32 R15 register
	undefined	
	nop	
	raz_wi	
	execute	
	assert	
unpredictableModal	Enumeration	Specify behavior for UNPREDICTABLE instructions in certain AArch32 modes (for example, MRS using SPSR in System mode)
	undefined	
	nop	
	assert	
maxSIMDUnroll	Uns32	If SIMD operations are supported, specify the maximum number of parallel SIMD operations to unroll (unrolled operations can be faster, but produce more verbose JIT code)
override_debugMask	Uns32	Specifies debug mask, enabling debug output for model components
ASIDCacheSize	Uns32	Specifies the number of different ASIDs for which TLB entries are cached; a value of 0 implies no limit

thumbNoCond	Boolean	Specify whether trapped Thumb instructions set CV=1 and COND field in syndrome (if False, both are zero)
enableHostAtomics	Boolean	Enable use of host atomic instructions to emulate load/store exclusive operations (not architecturally accurate, accelerates parallel simulation)
override_fcsePresent	Boolean	Specifies that FCSE is present (if true)
override_fpexcDexPresent	Boolean	Specifies that the FPEXC.DEX register field is implemented (if true)
override_advSIMDPresent	Boolean	Specifies that Advanced SIMD extensions are present (if true)
override_vfpPresent	Boolean	Specifies that VFP extensions are present (if true)
override_SCTLR_V	Boolean	Override SCTLR.V with the passed value (enables high vectors; also configurable using VINITHI pin)
override_SCTLR_IE	Boolean	Override SCTLR.IE with the passed value (configures instruction endianness; also configurable using CFGIE pin)
override_SCTLR_EE	Boolean	Override SCTLR.EE with the passed value (configures exception data endianness; also configurable using CFGEE pin)
override_SCTLR_TE	Boolean	Override SCTLR.TE with the passed value (configures Thumb state for exception handling; also configurable using TEINIT pin)
override_SCTLR_NMFI	Boolean	Override SCTLR.NMFI with the passed value (configures NMFI state for exception handling; also configurable using CFGNMFI pin)
override_SCTLR_CP15BEN_Present	Boolean	Enable ARMv7 SCTLR.CP15BEN bit (CP15 barrier enable)
override_MIDR	Uns32	Override MIDR/MIDR_EL1 register
override_CTR	Uns32	Override CTR/CTR_EL0 register
override_TLBTR	Uns32	Override TLBTR register
override_MPUIR	Uns32	Override MPUIR register
override_MPUIR_DRegion	Uns32	Override MPUIR.DRegion with the passed value
override_CLIDR	Uns32	Override CLIDR/CLIDR_EL1 register
override_TCMTR	Uns32	Override TCMTR register
override_AIDR	Uns32	Override AIDR/AIDR_EL1 register
override_ATCMRR	Uns32	Override ATCMRR register (specifies ATCM base address and size)
override_ITCMRR_E	Boolean	Override ITCMRR register enable bit
override_ITCMRR_SIZE	Uns32	Override ITCMRR register size bits
override_tcmsPresent	Boolean	Specifies that TCM is present (if true)
override_BTCMRR	Uns32	Override BTCMRR register (specifies BTCM base address and size)
override_DTCMRR_SIZE	Uns32	Override DTCMRR register size bits
override_PFR0	Uns32	Override ID_PFR0/ID_PFR0_EL1 register
override_PFR1	Uns32	Override ID_PFR1/ID_PFR1_EL1 register
override_DFR0	Uns32	Override ID_DFR0/ID_DFR0_EL1 register
override_AFR0	Uns32	Override ID_AFR0/ID_AFR0_EL1 register
override_MMFR0	Uns32	Override ID_MMFR0/ID_MMFR0_EL1 register
override_MMFR1	Uns32	Override ID_MMFR1/ID_MMFR1_EL1 register
override_MMFR2	Uns32	Override ID_MMFR2/ID_MMFR2_EL1 register
override_MMFR3	Uns32	Override ID_MMFR3/ID_MMFR3_EL1 register
override_ISAR0	Uns32	Override ID_ISAR0/ID_ISAR0_EL1 register
override_ISAR1	Uns32	Override ID_ISAR1/ID_ISAR1_EL1 register
override_ISAR2	Uns32	Override ID_ISAR2/ID_ISAR2_EL1 register
override_ISAR3	Uns32	Override ID_ISAR3/ID_ISAR3_EL1 register
override_ISAR4	Uns32	Override ID_ISAR4/ID_ISAR4_EL1 register
override_ISAR5	Uns32	Override ID_ISAR5/ID_ISAR5_EL1 register

override_PMCRR	Uns32	Override PMCR/PMCR_EL0 register (not functionally significant in the model)
override_PMCEID0	Uns64	Override PMCEID0/PMCEID0_EL0 register (not functionally significant in the model)
override_PMCEID1	Uns64	Override PMCEID1/PMCEID1_EL0 register (not functionally significant in the model)
override_SACTLR	Uns32	Override Cortex-R profile SACTLR register (not functionally significant in the model)
override_BuildOptions0	Uns32	Override Cortex-R profile BuildOptions0 register (not functionally significant in the model)
override_BuildOptions1	Uns32	Override Cortex-R profile BuildOptions1 register (not functionally significant in the model)
override_DBGDIDR	Uns32	Override DBGDIDR register (not functionally significant in the model)
override_FPSID	Uns32	Override SIMD/VFP FPSID register
override_MVFR0	Uns32	Override SIMD/VFP MVFR0/MVFR0_EL1 register
override_MVFR1	Uns32	Override SIMD/VFP MVFR1/MVFR1_EL1 register
override_FPEXC	Uns32	Override SIMD/VFP FPEXC/FPEXC32_EL2 register
override_ERG	Uns32	Specifies exclusive reservation granule
override_CCSIDR_1I	Uns64	Override CCSIDR/CCSIDR_EL1 (level 1 instruction)
override_CCSIDR_1D	Uns64	Override CCSIDR/CCSIDR_EL1 (level 1 data)
override_CCSIDR_2I	Uns64	Override CCSIDR/CCSIDR_EL1 (level 2 instruction)
override_CCSIDR_2D	Uns64	Override CCSIDR/CCSIDR_EL1 (level 2 data)
override_CCSIDR_3I	Uns64	Override CCSIDR/CCSIDR_EL1 (level 3 instruction)
override_CCSIDR_3D	Uns64	Override CCSIDR/CCSIDR_EL1 (level 3 data)
override_CCSIDR_4I	Uns64	Override CCSIDR/CCSIDR_EL1 (level 4 instruction)
override_CCSIDR_4D	Uns64	Override CCSIDR/CCSIDR_EL1 (level 4 data)
override_CCSIDR_5I	Uns64	Override CCSIDR/CCSIDR_EL1 (level 5 instruction)
override_CCSIDR_5D	Uns64	Override CCSIDR/CCSIDR_EL1 (level 5 data)
override_CCSIDR_6I	Uns64	Override CCSIDR/CCSIDR_EL1 (level 6 instruction)
override_CCSIDR_6D	Uns64	Override CCSIDR/CCSIDR_EL1 (level 6 data)
override_CCSIDR_7I	Uns64	Override CCSIDR/CCSIDR_EL1 (level 7 instruction)
override_CCSIDR_7D	Uns64	Override CCSIDR/CCSIDR_EL1 (level 7 data)
override_STRoffsetPC12	Boolean	Specifies that STR/STR of PC should do so with 12:byte offset from the current instruction (if true), otherwise an 8:byte offset is used
override_mpuV5ExtAP	Boolean	Specifies that ARMv5 MPU extended access permissions implemented (cp15/5/2 and cp15/5/2)
override_fcseRequiresMMU	Boolean	Specifies that FCSE is active only when MMU is enabled (if true)
override_ignoreBadCp15	Boolean	Specifies whether invalid coprocessor 15 access should be ignored (if true) or cause Invalid Instruction exceptions (if false)
override_SGIDisable	Boolean	Override whether GIC SGIs may be disabled (if true) or are permanently enabled (if false)
override_condUndefined	Boolean	Force undefined instructions to take Undefined Instruction exception even if they are conditional
override_deviceStrongAligned	Boolean	Force accesses to Device and Strongly Ordered regions to be aligned
override_Control_V	Boolean	Override SCTLR.V with the passed value (deprecated, use override_SCTLR_V)
override_MainId	Uns32	Override MIDR register (deprecated, use override_MIDR)
override_CacheType	Uns32	Override CTR register (deprecated, use override_CTR)
override_TLBType	Uns32	Override TLBTR register (deprecated, use override_TLBTR)

override MPUType	Uns32	Override MPUIR register (deprecated, use override_MPUIR)
override.InstructionAttributes0	Uns32	Override ID_ISAR0 register (deprecated, use override_ISAR0)
override.InstructionAttributes1	Uns32	Override ID_ISAR1 register (deprecated, use override_ISAR1)
override.InstructionAttributes2	Uns32	Override ID_ISAR2 register (deprecated, use override_ISAR2)
override.InstructionAttributes3	Uns32	Override ID_ISAR3 register (deprecated, use override_ISAR3)
override.InstructionAttributes4	Uns32	Override ID_ISAR4 register (deprecated, use override_ISAR4)
override.InstructionAttributes5	Uns32	Override ID_ISAR5 register (deprecated, use override_ISAR5)

Table 8.1: Parameters that can be set in: CPU

8.1 Parameter values and limits

These are the formal parameter limits and actual parameter values

Name	Min	Max	Default	Actual
(Others)				
variant				Cortex-R4F
verbose			t	t
suppressCPSWarnings			f	f
showHiddenRegs			f	f
UAL			t	t
enableVFPAAtReset			f	f
useInternalTCMs			f	f
enableSystemBus			f	f
enableSystemMonitorBus			f	f
compatibility			ISA	ISA
unpredictableR15			undefined	undefined
unpredictableModal			nop	nop
maxSIMDUnroll	1	16	2	2
override_debugMask	0	0xffffffff	0	0
ASIDCacheSize	0	0x100	8	8
thumbNoCond			f	f
enableHostAtomics			f	f
endian			none	none
override_fcsePresent			f	f
override_fpexcDexPresent			f	f
override_advSIMDPresent			f	f
override_vfpPresent			f	f
override_SCTLR_V			f	f
override_SCTLR_IE			f	f
override_SCTLR_EE			f	f

override_SCTLR_TE			f	f
override_SCTLR_NMFI			f	f
override_SCTLR_CP15BEN_Present			f	f
override_MIDR	0	0xffffffff	0x411fc144	0x411fc144
override_CTR	0	0xffffffff	0x8003c003	0x8003c003
override_TLBTR	0	0xffffffff	0	0
override_MPUIR	0	0xffffffff	0x800	0x800
override_MPUIR_DRegion	0	0xffffffff	8	8
override_CLIDR	0	0xffffffff	0x9000003	0x9000003
override_TCMTR	0	0xffffffff	0x10001	0x10001
override_AIDR	0	0xffffffff	0	0
override_ATCMRR	0	0xffffffff	16	16
override_ITCMRR_E			f	f
override_ITCMRR_SIZE	0	0xffffffff	4	4
override_tcmsPresent			f	f
override_BTCMRR	0	0xffffffff	0x800010	0x800010
override_DTCMRR_SIZE	0	0xffffffff	4	4
override_PFR0	0	0xffffffff	0x131	0x131
override_PFR1	0	0xffffffff	1	1
override_DFR0	0	0xffffffff	0	0
override_AFR0	0	0xffffffff	0	0
override_MMFR0	0	0xffffffff	0x210030	0x210030
override_MMFR1	0	0xffffffff	0	0
override_MMFR2	0	0xffffffff	0x1200000	0x1200000
override_MMFR3	0	0xffffffff	0x211	0x211
override_ISAR0	0	0xffffffff	0x1101111	0x1101111
override_ISAR1	0	0xffffffff	0x13112111	0x13112111
override_ISAR2	0	0xffffffff	0x21232131	0x21232131
override_ISAR3	0	0xffffffff	0x1112131	0x1112131
override_ISAR4	0	0xffffffff	0x10142	0x10142
override_ISAR5	0	0xffffffff	0	0
override_PMCR	0	0xffffffff	0x41141800	0x41141800
override_PMCEID0	0	0xffffffffffffff	0	0
override_PMCEID1	0	0xffffffffffffff	0	0
override_SACTLR	0	0xffffffff	0x400000	0x400000
override_BuildOptions0	0	0xffffffff	0	0
override_BuildOptions1	0	0xffffffff	0x810	0x810
override_DBGDIDR	0	0xffffffff	0	0
override_FPSID	0	0xffffffff	0x41023148	0x41023148
override_MVFR0	0	0xffffffff	0x10110221	0x10110221
override_MVFR1	0	0xffffffff	17	17
override_FPEXC	0	0xffffffff	0	0
override_ERG	3	11	3	3
override_CCSIDR_1I	0	0xffffffffffffff	0xf03fe019	0xf03fe019
override_CCSIDR_1D	0	0xffffffffffffff	0xf03fe019	0xf03fe019

override_CCSIDR_2I	0	0xffffffffffff	0	0
override_CCSIDR_2D	0	0xffffffffffff	0	0
override_CCSIDR_3I	0	0xffffffffffff	0	0
override_CCSIDR_3D	0	0xffffffffffff	0	0
override_CCSIDR_4I	0	0xffffffffffff	0	0
override_CCSIDR_4D	0	0xffffffffffff	0	0
override_CCSIDR_5I	0	0xffffffffffff	0	0
override_CCSIDR_5D	0	0xffffffffffff	0	0
override_CCSIDR_6I	0	0xffffffffffff	0	0
override_CCSIDR_6D	0	0xffffffffffff	0	0
override_CCSIDR_7I	0	0xffffffffffff	0	0
override_CCSIDR_7D	0	0xffffffffffff	0	0
override_STRoffsetPC12			t	t
override_mpuV5ExtAP			f	f
override_fcseRequiresMMU			f	f
override_ignoreBadCp15			f	f
override_SGIDisable			f	f
override_condUndefined			f	f
override_deviceStrongAligned			f	f
override_Control_V			f	f
override_MainId	0	0xffffffff	0x411fc144	0x411fc144
override_CacheType	0	0xffffffff	0x8003c003	0x8003c003
override_TLBType	0	0xffffffff	0	0
override MPUType	0	0xffffffff	0x800	0x800
override_InstructionAttributes0	0	0xffffffff	0x1101111	0x1101111
override_InstructionAttributes1	0	0xffffffff	0x13112111	0x13112111
override_InstructionAttributes2	0	0xffffffff	0x21232131	0x21232131
override_InstructionAttributes3	0	0xffffffff	0x1112131	0x1112131
override_InstructionAttributes4	0	0xffffffff	0x10142	0x10142
override_InstructionAttributes5	0	0xffffffff	0	0

Table 8.2: Parameter values and limits

Chapter 9

Execution Modes

Mode	Code
User	16
FIQ	17
IRQ	18
Supervisor	19
Abort	23
Undefined	27
System	31

Table 9.1: Modes implemented in: CPU

Chapter 10

Exceptions

Exception	Code
Reset	0
Undefined	1
SupervisorCall	2
PrefetchAbort	5
DataAbort	6
IRQ	8
FIQ	9

Table 10.1: Exceptions implemented in: CPU

Chapter 11

Hierarchy of the model

A CPU core may be configured to instance many processors of a Symmetrical Multi Processor (SMP). A CPU core may also have sub elements within a processor, for example hardware threading blocks.

OVP processor models can be written to include SMP blocks and to have many levels of hierarchy. Some OVP CPU models may have a fixed hierarchy, and some may be configured by settings in a configuration register. Please see the register definitions of this model.

This model documentation shows the settings and hierarchy of the default settings for this model variant.

11.1 Level 1: CPU

This level in the model hierarchy has 4 commands.

This level in the model hierarchy has 12 register groups:

Group name	Registers
Core	16
Control	3
User	7
FIQ	8
IRQ	3
Supervisor	3
Undefined	3
Abort	3
SIMD_VFP	16
SIMD_VFP_SYS	5
Coprocessor_32_bit	88
Integration_support	5

Table 11.1: Register groups

This level in the model hierarchy has no children.

Chapter 12

Model Commands

A Processor model can implement one or more **Model Commands** available to be invoked from the simulator command line, from the OP API or from the Imperas Multiprocessor Debugger.

12.1 Level 1: CPU

12.1.1 debugflags

show or modify the processor debug flags

Argument	Type	Description
-get	Boolean	print current processor flags value
-mask	Boolean	print valid debug flag bits
-set	Int32	new processor flags (only flags 0x000003e4 can be modified)

Table 12.1: debugflags command arguments

12.1.2 isync

specify instruction address range for synchronous execution

Argument	Type	Description
-addresshi	Uns64	end address of synchronous execution range
-addresslo	Uns64	start address of synchronous execution range

Table 12.2: isync command arguments

12.1.3 itrace

enable or disable instruction tracing

Argument	Type	Description
-access	String	show memory accesses by this instruction. Argument can be any combination of X (execute), A (load or store access) and S (system)

-after	Uns64	apply after this many instructions
-enable	Boolean	enable instruction tracing
-full	Boolean	turn on all trace features
-instructioncount	Boolean	include the instruction number in each trace
-memory	String	(Alias for access). show memory accesses by this instruction. Argument can be any combination of X (execute), A (load or store access) and S (system)
-mode	Boolean	show processor mode changes
-off	Boolean	disable instruction tracing
-on	Boolean	enable instruction tracing
-processorname	Boolean	Include processor name in all trace lines
-registerchange	Boolean	show registers changed by this instruction
-registers	Boolean	show registers after each trace

Table 12.3: itrace command arguments

12.1.4 listSysRegsAA32

12.1.4.1 Argument description

List all AArch32 system registers

Chapter 13

Registers

13.1 Level 1: CPU

13.1.1 Core

Registers at level:1, type:CPU group:Core

Name	Bits	Initial-Hex	RW	Description
r0	32	0	rw	
r1	32	0	rw	
r2	32	0	rw	
r3	32	0	rw	
r4	32	0	rw	
r5	32	0	rw	
r6	32	0	rw	
r7	32	0	rw	
r8	32	0	rw	
r9	32	0	rw	
r10	32	0	rw	
r11	32	0	rw	frame pointer
r12	32	0	rw	
sp	32	0	rw	stack pointer
lr	32	0	rw	
pc	32	0	rw	program counter

Table 13.1: Registers at level 1, type:CPU group:Core

13.1.2 Control

Registers at level:1, type:CPU group:Control

Name	Bits	Initial-Hex	RW	Description
fps	32	0	rw	archaic FPSCR view (for gdb)
cpsr	32	1d3	rw	
spsr	32	0	rw	

Table 13.2: Registers at level 1, type:CPU group:Control

13.1.3 User

Registers at level:1, type:CPU group:User

Name	Bits	Initial-Hex	RW	Description
r8_usr	32	0	rw	
r9_usr	32	0	rw	
r10_usr	32	0	rw	
r11_usr	32	0	rw	
r12_usr	32	0	rw	
sp_usr	32	0	rw	
lr_usr	32	0	rw	

Table 13.3: Registers at level 1, type:CPU group:User

13.1.4 FIQ

Registers at level:1, type:CPU group:FIQ

Name	Bits	Initial-Hex	RW	Description
r8_fiq	32	0	rw	
r9_fiq	32	0	rw	
r10_fiq	32	0	rw	
r11_fiq	32	0	rw	
r12_fiq	32	0	rw	
sp_fiq	32	0	rw	
lr_fiq	32	0	rw	
spsr_fiq	32	0	rw	

Table 13.4: Registers at level 1, type:CPU group:FIQ

13.1.5 IRQ

Registers at level:1, type:CPU group:IRQ

Name	Bits	Initial-Hex	RW	Description
sp_irq	32	0	rw	
lr_irq	32	0	rw	
spsr_irq	32	0	rw	

Table 13.5: Registers at level 1, type:CPU group:IRQ

13.1.6 Supervisor

Registers at level:1, type:CPU group:Supervisor

Name	Bits	Initial-Hex	RW	Description
sp_svc	32	0	rw	
lr_svc	32	0	rw	
spsr_svc	32	0	rw	

Table 13.6: Registers at level 1, type:CPU group:Supervisor

13.1.7 Undefined

Registers at level:1, type:CPU group:Undefined

Name	Bits	Initial-Hex	RW	Description
------	------	-------------	----	-------------

sp_undef	32	0	rw	
lr_undef	32	0	rw	
spsr_undef	32	0	rw	

Table 13.7: Registers at level 1, type:CPU group:Undefined

13.1.8 Abort

Registers at level:1, type:CPU group:Abort

Name	Bits	Initial-Hex	RW	Description
sp_abt	32	0	rw	
lr_abt	32	0	rw	
spsr_abt	32	0	rw	

Table 13.8: Registers at level 1, type:CPU group:Abort

13.1.9 SIMD_VFP

Registers at level:1, type:CPU group:SIMD_VFP

Name	Bits	Initial-Hex	RW	Description
d0	64	0	rw	
d1	64	0	rw	
d2	64	0	rw	
d3	64	0	rw	
d4	64	0	rw	
d5	64	0	rw	
d6	64	0	rw	
d7	64	0	rw	
d8	64	0	rw	
d9	64	0	rw	
d10	64	0	rw	
d11	64	0	rw	
d12	64	0	rw	
d13	64	0	rw	
d14	64	0	rw	
d15	64	0	rw	

Table 13.9: Registers at level 1, type:CPU group:SIMD_VFP

13.1.10 SIMD_VFP_SYS

Registers at level:1, type:CPU group:SIMD_VFP_SYS

Name	Bits	Initial-Hex	RW	Description
FPSID	32	41023148	r-	floating-point system ID
FPSCR	32	0	rw	floating-point status/control
FPEXC	32	0	rw	floating-point exception
MVFR0	32	10110221	r-	Media/VFP feature 0
MVFR1	32	11	r-	Media/VFP feature 1

Table 13.10: Registers at level 1, type:CPU group:SIMD_VFP_SYS

13.1.11 Coprocessor_32_bit

Registers at level:1, type:CPU group:Coprocessor_32_bit

Name	Bits	Initial-Hex	RW	Description
ACTLR	32	20	rw	Auxiliary Control
ADFSR	32	0	rw	Auxiliary Data Fault Status
AIFSR	32	0	rw	Auxiliary Instruction Fault Status
ATCMRR	32	10	rw	ATCM Region
BPIALL	32	-	-w	Branch Predictor Invalidate All
BPIMVA	32	-	-w	Branch Predictor Invalidate by VA
BTCMRR	32	10	rw	BTCM Region
BuildOptions0	32	0	r-	Build Options 0
BuildOptions1	32	810	r-	Build Options 1
CCSIDR	32	f03fe019	r-	Cache Size ID
CFLR	32	0	rw	Correctable Fault Location
CLIDR	32	9000003	r-	Cache Level ID
CONTEXTIDR	32	0	rw	Context ID
CP15DMB	32	-	-w	CP15 Data Memory Barrier
CP15DSB	32	-	-w	CP15 Data Synchronization Barrier
CP15ISB	32	-	-w	CP15 Instruction Synchronization Barrier
CP15NOP	32	-	-w	CP15 NOP
CPACR	32	c0000000	rw	Coprocessor Access Control
CSOR	32	-	-w	Cache Size Override
CSSELR	32	0	rw	Cache Size Selection
CTR	32	8003c003	r-	Cache Type
DBGDIDR	32	0	r-	Debug ID
DCCIMVAC	32	-	-w	Data Cache Line Clean and Invalidate by VA to PoC
DCCISW	32	-	-w	Data Cache Line Clean and Invalidate by Set/Way
DCCMVAC	32	-	-w	Data Cache Line Clean by VA to PoC
DCCMVAU	32	-	-w	Data Cache Line Clean by VA to PoU
DCCSW	32	-	-w	Data Cache Line Clean by Set/Way
DCIALL	32	-	-w	Data Cache Invalidate All
DCIMVAC	32	-	-w	Data Cache Line Invalidate by VA to PoC
DCISW	32	-	-w	Data Cache Line Invalidate by Set/Way
DFAR	32	0	rw	Data Fault Address
DFSR	32	0	rw	Data Fault Status
DRACR	32	0	rw	Data Region Access Control
DRBAR	32	0	rw	Data Region Base Address
DRSR	32	0	rw	Data Region Size/Enable
ICIALLU	32	-	-w	Instruction Cache Invalidate All
ICIMVAU	32	-	-w	Instruction Cache Invalidate by VA
ID.AFR0	32	0	r-	Auxiliary Feature 0
ID.DFR0	32	0	r-	Debug Feature 0
ID.ISAR0	32	1101111	r-	Instruction Set Attribute 0
ID.ISAR1	32	13112111	r-	Instruction Set Attribute 1
ID.ISAR2	32	21232131	r-	Instruction Set Attribute 2
ID.ISAR3	32	1112131	r-	Instruction Set Attribute 3
ID.ISAR4	32	10142	r-	Instruction Set Attribute 4
ID.ISAR5	32	0	r-	Instruction Set Attribute 5
ID.MMFR0	32	210030	r-	Memory Model Feature 0
ID.MMFR1	32	0	r-	Memory Model Feature 1
ID.MMFR2	32	1200000	r-	Memory Model Feature 2
ID.MMFR3	32	211	r-	Memory Model Feature 3
ID.PFR0	32	131	r-	Processor Feature 0
ID.PFR1	32	1	r-	Processor Feature 1
IFAR	32	0	rw	Instruction Fault Address

IFSR	32	0	rw	Instruction Fault Status
JIDR	32	0	rw	Jazelle ID
JMCR	32	0	rw	Jazelle Main Configuration
JOSCR	32	0	rw	Jazelle OS Control
MIDR	32	411fc144	r-	Main ID
MPIDR	32	0	r-	Multiprocessor Affinity
MPUIR	32	800	r-	MPU Type
PAR	32	0	rw	Physical Address
PMCCNTR	32	0	rw	Performance Monitors Cycle Count
PMCNTENCLR	32	0	rw	Performance Monitors Count Enable Clear
PMCNTENSET	32	0	rw	Performance Monitors Count Enable Set
PMCR	32	41141800	rw	Performance Monitors Control
PMINTENCLR	32	0	rw	Performance Monitors Interrupt Enable Clear
PMINTENSET	32	0	rw	Performance Monitors Interrupt Enable Set
PMOVSr	32	0	rw	Performance Monitors Overflow Flag Status
PMSELR	32	0	rw	Performance Monitors Event Counter Selection
PMSWINC	32	-	-w	Performance Monitors Software Increment
PMUSERENR	32	0	rw	Performance Monitors User Enable
PMXEVCNTR	32	0	rw	Performance Monitors Selected Event Count
PMXEVTYPER	32	0	rw	Performance Monitors Selected Event Type
RGNR	32	0	rw	Region Number
SACTLR	32	400000	rw	Secondary Auxiliary Control
SCTLR	32	e50878	rw	System Control
SPCR	32	0	rw	Slave Port Control
TCMTR	32	10001	r-	TCM Type
TPIDRPRW	32	0	rw	PL0 Read/Write Software Thread ID
TPIDRURO	32	0	rw	PL0 Read-Only Software Thread ID
TPIDRURW	32	0	rw	PL1 Software Thread ID
VALDEBUGENCLR	32	0	rw	VAL Debug Request Enable Clear
VALDEBUGENSET	32	0	rw	VAL Debug Request Enable Set
VALFIQENCLR	32	0	rw	nVAL FIQ Enable Clear
VALFIQENSET	32	0	rw	nVAL FIQ Enable Set
VALIRQENCLR	32	0	rw	nVAL IRQ Enable Clear
VALIRQENSET	32	0	rw	nVAL IRQ Enable Set
VALRESETENCLR	32	0	rw	nVAL Reset Enable Clear
VALRESETENSET	32	0	rw	nVAL Reset Enable Set

Table 13.11: Registers at level 1, type:CPU group:Coprocessor_32_bit

13.1.12 Integration_support

Registers at level:1, type:CPU group:Integration_support

Name	Bits	Initial-Hex	RW	Description
transactPL	32	1	r-	privilege level of current memory transaction
transactAT	32	0	r-	current memory transaction type: PA=1, VA=0
HaltReason	8	0	r-	bit field indicating halt reason
atomicType	8	0	r-	current atomic instruction type (1:atomic, 2:exclusive)
WakeWFINow	8	0	-w	wake up core from WFI state

Table 13.12: Registers at level 1, type:CPU group:Integration_support